

Національний технічний університет України  
«Київський політехнічний інститут імені Ігоря Сікорського»  
Радіотехнічний факультет  
Кафедра радіотехнічних систем

## **ЗВІТ**

**до лабораторної роботи № 6**

з дисципліни «Програмовані засоби в інтелектуальній  
радіoeлектронній техніці»

*на тему:*

**«СТВОРЕННЯ ПЕРШОГО ПРОЕКТУ НА МОВІ C»**

Виконав:

студент групи РЕ-41

Кравченко Артем

Викладач: \_\_\_\_\_

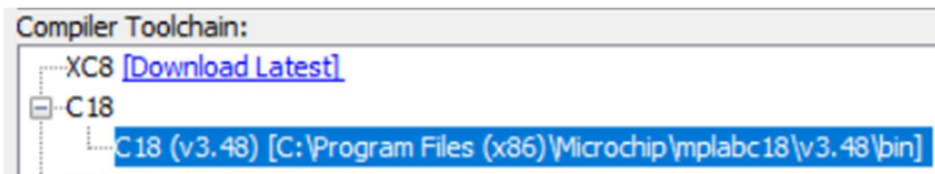
Київ – 2026

## Мета роботи

Ознайомитися з особливостями створення проєкту на мові C для мікроконтролера PIC18F4550 у середовищі MPLAB з використанням компілятора C18, а також перевірити роботу програми на макеті та в режимі симуляції.

## План виконання роботи

1. Створено новий проєкт для мікроконтролера PIC18F4550 у MPLAB та обрано компілятор Microchip C18.



2. До проєкту підключено файл сценарію компонування, який враховує наявність програмного завантажувача у пам'яті мікроконтролера.

```
15  LIBPATH .
16
17  FILES c018i.o
18  FILES clib.lib
19  FILES pl8f4550.lib
20
21  CODEPAGE  NAME=bootloader  START=0x0          END=0xFFF      PROTECTED
22  CODEPAGE  NAME=vectors     START=0x1000    END=0x1029     PROTECTED
23  CODEPAGE  NAME=page        START=0x102A    END=0x7FFF     PROTECTED
24  CODEPAGE  NAME=idlocs      START=0x200000  END=0x200007   PROTECTED
25  CODEPAGE  NAME=config      START=0x300000  END=0x30000D   PROTECTED
26  CODEPAGE  NAME=devid       START=0x3FFFFE  END=0x3FFFFF   PROTECTED
27  CODEPAGE  NAME=eedata      START=0xF00000  END=0xF000FF   PROTECTED
28
29  ACCESSBANK NAME=accessram  START=0x0       END=0x5F
30  DATABANK   NAME=gpr0       START=0x60      END=0xFF
31  DATABANK   NAME=gpr1       START=0x100     END=0x1FF
32  DATABANK   NAME=gpr2       START=0x200     END=0x2FF
33  DATABANK   NAME=gpr3       START=0x300     END=0x3FF
34  DATABANK   NAME=usb4       START=0x400     END=0x4FF      PROTECTED
35  DATABANK   NAME=usb5       START=0x500     END=0x5FF      PROTECTED
36  DATABANK   NAME=usb6       START=0x600     END=0x6FF      PROTECTED
37  DATABANK   NAME=usb7       START=0x700     END=0x7FF      PROTECTED
38  ACCESSBANK NAME=accesssfr  START=0xF60     END=0xFFFF     PROTECTED
39
40  SECTION    NAME=CONFIG     ROM=config
41
42  STACK SIZE=0x100 RAM=gpr3
43
44  SECTION    NAME=USB_VARS   RAM=usb4
```

3. Створено файл основної програми на мові C та реалізовано спільний приклад виконання лабораторної роботи (без поділу на окремі case).

```

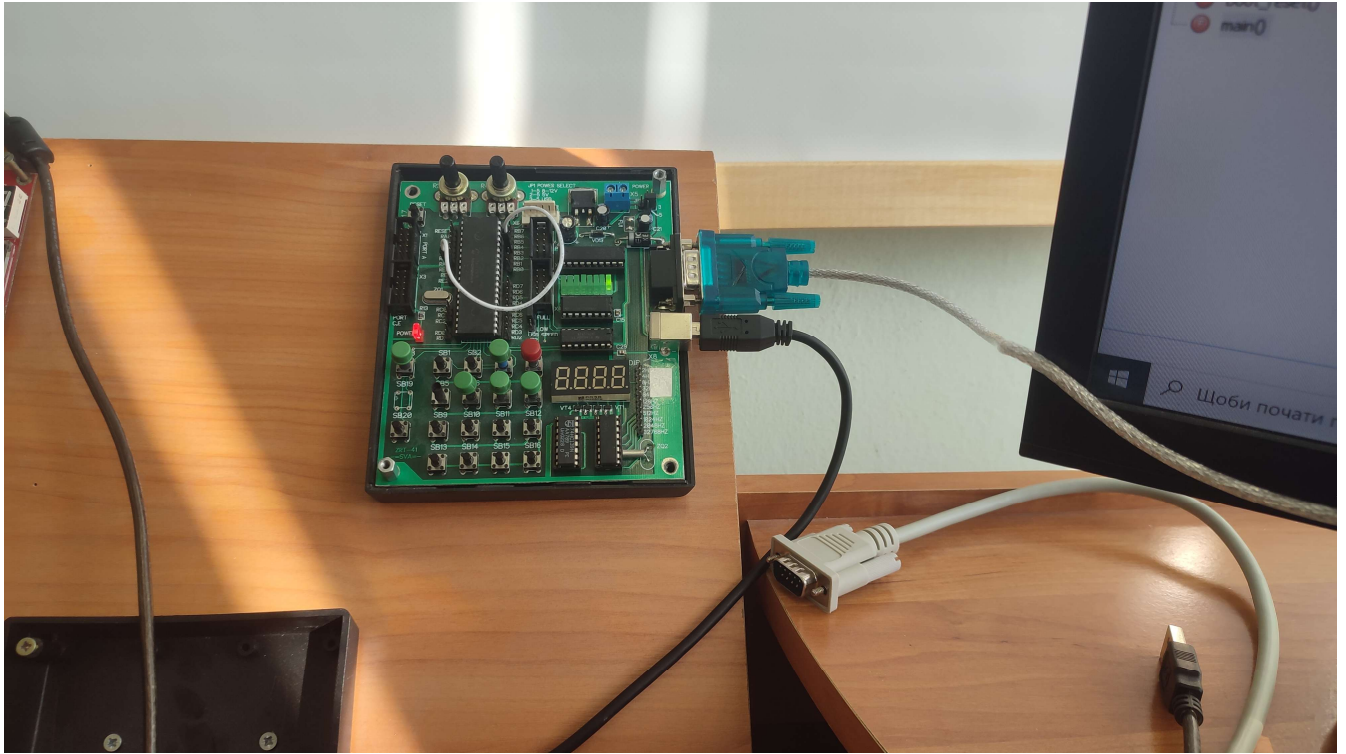
1  #include <pl8f4550.h>
2  #include <delays.h>
3
4  unsigned int counter;
5  unsigned int adc_value;
6  unsigned char buffer[6];
7  unsigned char k;
8
9  void USARTInit(void)
10 {
11     SPBRGH = 0;
12     BAUDCONbits.BRG16 = 0;
13
14     SPBRG = 77;
15     TXSTAbits.BRGH = 0;
16     TXSTAbits.SYNC = 0;
17     RCSTAbits.SPEN = 1;
18
19     TXSTAbits.TX9 = 0;
20     TXSTAbits.TXEN = 1;
21 }
22
23 void ADCInit(void)
24 {
25     ADCON0 = 0b00000001;
26     ADCON1 = 0b00001110;
27     ADCON2 = 0b00101110;
28
29     ADCON0bits.GO = 1;
30 }
31
32 void sendCharUSART(unsigned char data)
33 {
34     while(TXSTAbits.TRMT == 0);
35     TXREG = data;
36 }
37
38 void sendWordUSART(unsigned char* word)
39 {
40     int i = 0;
41     sendCharUSART('\f');
42     while(word[i] != '\0' && i != 5)
43     {
44         sendCharUSART(word[i]);
45         i++;
46     }
47     sendCharUSART('m');
48     sendCharUSART('v');
49     // sendCharUSART('\r');
50 }
51
52 void numberToWord(unsigned int number, unsigned char* word)
53 {
54     register unsigned char n;
55     for(n = 0; n <= 4; n++)
56     {
57         word[n] = '0';
58     }
59     for(n = 0; n <= 4; n++)
60     {
61         word[4-n] = (number % 10) | '0';
62         number /= 10;
63         if (!number) break;
64     }
65 }
66
67 #pragma code main=0x102A
68 void main(void)
69 {
70     TRISB = 0;
71     ADCON1 = 0x0F;
72     counter = 1;
73     adc_value = 0;
74
75     USARTInit();
76     ADCInit();
77
78     while (1)
79     {
80         if (!ADCON0bits.GO)
81         {
82             adc_value = ADRESH;
83             PIR1bits.ADIF = 0;
84             ADCON0bits.GO = 1;
85         }
86         PORTB = counter;
87         counter = counter << 1;
88         if (counter == 256) counter = 1;
89         adc_value = adc_value * 39;
90         adc_value = adc_value / 2;
91         numberToWord(adc_value, buffer);
92         sendWordUSART(buffer);
93         for (k = 0; k < 1; k++)
94         {
95             Delay10KTCYx(250);
96         }
97     }
98 }

```

4. Виконано налагодження програми, перевірено правильність компіляції та запуску коду з урахуванням зміщення стартової адреси.

```
BUILD SUCCESSFUL (total time: 555ms)
Loading code from D:/Artem/IT/MPAsm_Projects/Lab6_KravchenkoArtem_RE-41/dist/default/production/Lab6_KravchenkoArtem_RE-41.production.hex..
Program loaded with pack, PIC18Fxxxx_DFP, 1.7.171, Microchip
Loading completed
```

5. Проведено тестування роботи програми на навчальному макеті.



## **Відповіді на контрольні запитання**

### **1. Як впливає на програму наявність програмного загрузчика?**

Програмний загрузчик займає частину пам'яті програм, тому основний код не можна розміщувати з нульової адреси. Через це початкова адреса програми зміщується, а це треба враховувати у налаштуванні проєкту.

### **2. Яке призначення сценарію компонування проєкту?**

Сценарій компонування визначає, як саме код і дані розміщуються у пам'яті мікроконтролера. Він не допускає накладання основної програми на область, зайняту bootloader.

### **3. Чому у дизасембльованому лістингу проста програма на C реалізована значною кількістю команд на асемблері?**

Тому що компілятор додає службові дії: ініціалізацію пам'яті, стеку, змінних і виклики бібліотечних функцій. Через це навіть проста конструкція на C після компіляції перетворюється на багато асемблерних команд.

### **4. Якими командами на асемблері можна реалізувати оператор while, зокрема while(TXSTAbits.TRMT == 0);?**

Такий цикл реалізують командами перевірки біта і переходу, наприклад BTFSS або BTFSC разом із GOTO чи BRA. Поки біт не набуде потрібного стану, програма повертається на початок перевірки.

### **5. Як на C реалізуються операції з бітами?**

Операції з бітами виконуються або через бітові оператори, або через звернення до окремих бітів регістрів, наприклад TXSTAbits.TXEN = 1. Це дозволяє зручно встановлювати, скидати або перевіряти окремі біти.

### **6. Поясніть механізм роботи з масивом buff[5] з використанням вказівників.**

У функцію передається адреса першого елемента масиву, тому зміни виконуються безпосередньо у самому масиві. Це дозволяє записувати символи в buff[5] без копіювання всього масиву.

## **7. Яке призначення має символ % у функції hex\_to\_ascii?**

Символ % означає остачу від ділення. У цій функції він використовується для виділення окремої десяткової цифри числа під час його перетворення у ASCII.

## **8. Навіщо у процедурі нормування результат АЦП спочатку множиться, а потім ділиться на 256?**

Так роблять для зручного цілочисельного обчислення без використання float. Спочатку значення масштабується множенням, а потім швидко повертається до потрібного діапазону діленням на 256, тобто зсувом праворуч на 8 біт.

## **Висновок**

У ході лабораторної роботи було створено та налагоджено проєкт на мові C для мікроконтролера PIC18F4550. Під час виконання роботи було закріплено навички створення проєкту в MPLAB, підключення сценарію компонування, запуску програми з урахуванням bootloader та перевірки її роботи у симуляторі й на макеті. Отримано практичні навички використання мови C для роботи з ресурсами мікроконтролера.